The big bolded letters are what I asked, and the regular font is the response the AI gave me.

I'm creating a story game where you need to escape a dungeon by collecting money and opening the vault to get a power up to escape.

```
from dataclasses import dataclass
from drafter import *

START_HOURS = 8
START_MONEY = 5
START_ROOM = "Entrance"
START_MESSAGE = "You stand at the dungeon entrance."
VAULT_COST = 15
POWER_UP_VALUE = 20
POWER_UP_BONUS = 5
LOOT_GOAL = 40
ROOMS = ["Crystal Cave", "Goblin Den", "Flooded Tunnel", "Vault"]

@dataclass
class Item:
name: str
description: str
value: int

@dataclass
class State:
player_name: str
hours_left: int
dungeon_room: str
money: int
inventory: list[Item]
message: str

def new_state() -> State:
return State("", START_HOURS, START_ROOM, START_MONEY, [], START_MESSAGE)

def make_item(room: str) -> Item:
```

```
if room == "Crystal Cave":
return Item("Glowing Crystal", "Hums with soft light.", 25)
elif room == "Goblin Den":
return Item("Goblin Dagger", "Chipped but still sharp.", 10)
elif room == "Flooded Tunnel":
return Item("Soggy Map", "Shows a shortcut out.", 5)
else:
return Item("Dragon Egg", "Warm and heavy.", 40)

def room_cost(room: str) -> int:
if room == "Crystal Cave":
return 2
elif room == "Goblin Den":
return 3
elif room == "Flooded Tunnel":
return 1
else:
return 2
```

this is what i have written so far. give me ideas of other def functions should I add for my story game to work.

You have a good start with the game's data and room rules. To make it playable, add functions for moving, collecting, spending, and winning or losing.

Here are some ideas:
- move_to_room(state, room) — Check whether the player has enough hours, subtract the room's cost, and update dungeon_room.
- collect_item(state) — Create the current room's item, add it to inventory, and add its value to money.
- open_vault(state) — Check whether the player has enough money to pay VAULT_COST; if so, unlock the vault and grant the power-up.
- use_power_up(state) — Apply the power-up bonus or mark the player as ready to escape.
- escape_dungeon(state) — Check that the player has the power-up and meets your escape requirements, then show a win message.
- check_game_over(state) — Decide whether the player has run out of hours or can no longer reach the goal.
- show_status(state) — Display the player's room, remaining hours, money, and inventory.
- restart_game() — Create and return a fresh state with new_state().

For example, a movement function could look like this:
def move_to_room(state: State, room: str) -> State:

```
    if room not in ROOMS:
        state.message = "You can't go there."
    elif state.hours_left < room_cost(room):
        state.message = "You don't have enough time to reach that room."
    else:
        state.hours_left -= room_cost(room)
        state.dungeon_room = room
        state.message = f"You enter the {room}."
    return state
```

One design choice to settle: does collecting an item add its value to money, or does the player need to sell the item separately? Your LOOT_GOAL constant could help define the escape requirement, while POWER_UP_VALUE and POWER_UP_BONUS could describe what happens after opening the vault.

I have a list of functions that I have named and what I want them to do, could you format how the code should look. I have them in order of helper functions organized by room, items, inventory/coins, endings, and then routes.

make_item(room): makes the hidden items in the room
room_cost(room): how many hours the room takes to explore
room_coins(room): coins acquired when searched
room_description(room): description of the room
room_lines(): list of rooms you can choose from

has_item(items, name): loops wheres it checks if item is in inventory or not
total_loot_value(items): adds up value of items acquired
count_power_ups(items): counts items if theyre worth more than 20
final_score(items): total value of items with additional if you have power ups
inventory_lines(items): puts items into inventory

pick_ending(state): decides if you have the requirements to win or not
ending_text(ending, name): what is shown with the ending you're given
ending_page(state, ending): ending page that shows your ending, score, and play again

Routes:
index(state): start screen
start_game(state, name): starts game when name is given
time_left(state): showing time left and if your game is still going or if you failed

explore(state): explores room
loot_room(state, take_item): calculates time and money spent/gained after exploring a room
view_items(state): shows inventory
leave_dungeon(state): shows your score when you leave
finale(state): shows game score and ending you got
restart(state): restarts game

Sure. Here's a code-shaped outline in your order. It shows what each function could look like and what it should return; you can fill in the page contents and exact game rules as you build them.

One note: inventory_lines(items) sounds like it should create display text, while adding items to inventory should happen in loot_room.

```
from dataclasses import dataclass
from drafter import *

START_HOURS = 8
START_MONEY = 5
START_ROOM = "Entrance"
START_MESSAGE = "You stand at the dungeon entrance."
VAULT_COST = 15
POWER_UP_VALUE = 20
POWER_UP_BONUS = 5
LOOT_GOAL = 40
ROOMS = ["Crystal Cave", "Goblin Den", "Flooded Tunnel", "Vault"]


@dataclass
class Item:
    name: str
    description: str
    value: int


@dataclass
class State:
    player_name: str
    hours_left: int
    dungeon_room: str
    money: int
    inventory: list[Item]
    message: str
```

```
    ending: str = ""


# Helper functions: rooms

def make_item(room: str) -> Item:
    """Return the item hidden in a room."""
    if room == "Crystal Cave":
        return Item("Glowing Crystal", "Hums with soft light.", 25)
    elif room == "Goblin Den":
        return Item("Goblin Dagger", "Chipped but still sharp.", 10)
    elif room == "Flooded Tunnel":
        return Item("Soggy Map", "Shows a shortcut out.", 5)
    else:
        return Item("Dragon Egg", "Warm and heavy.", 40)


def room_cost(room: str) -> int:
    """Return how many hours it takes to explore a room."""
    if room == "Crystal Cave":
        return 2
    elif room == "Goblin Den":
        return 3
    elif room == "Flooded Tunnel":
        return 1
    elif room == "Vault":
        return 2
    return 0


def room_coins(room: str) -> int:
    """Return the coins found when searching a room."""
    if room == "Crystal Cave":
        return 3
    elif room == "Goblin Den":
        return 8
    elif room == "Flooded Tunnel":
        return 2
    elif room == "Vault":
        return 0
    return 0


def room_description(room: str) -> str:
```

```
    """Return a description of a room."""
    descriptions = {
        "Entrance": "A cold passage leads deeper into the dungeon.",
        "Crystal Cave": "Glowing crystals cover the cave walls.",
        "Goblin Den": "A messy den is filled with goblin-sized furniture.",
        "Flooded Tunnel": "Shallow water covers the stone floor.",
        "Vault": "A heavy door protects the old vault.",
    }
    return descriptions.get(room, "You don't recognize this room.")


def room_lines() -> list[str]:
    """Return the rooms the player can choose from."""
    return ROOMS


# Helper functions: items and inventory

def has_item(items: list[Item], name: str) -> bool:
    """Return True if an item with this name is in the list."""
    for item in items:
        if item.name == name:
            return True
    return False


def total_loot_value(items: list[Item]) -> int:
    """Add together the values of all items."""
    total = 0
    for item in items:
        total += item.value
    return total


def count_power_ups(items: list[Item]) -> int:
    """Count items worth more than 20."""
    count = 0
    for item in items:
        if item.value > POWER_UP_VALUE:
            count += 1
    return count


def final_score(items: list[Item]) -> int:
```

```
    """Calculate the score, adding a bonus for each power-up."""
    return total_loot_value(items) + count_power_ups(items) * POWER_UP_BONUS


def inventory_lines(items: list[Item]) -> list[str]:
    """Create one display line for each item."""
    lines = []
    for item in items:
        lines.append(f"{item.name}: {item.description} (value: {item.value})")
    return lines


# Helper functions: endings

def pick_ending(state: State) -> str:
    """Choose an ending based on the player's progress."""
    if state.hours_left <= 0:
        return "ran_out_of_time"
    elif state.money >= VAULT_COST and has_item(state.inventory, "Vault Key"):
        return "escaped"
    else:
        return "left_without_escaping"


def ending_text(ending: str, name: str) -> str:
    """Return the text for the chosen ending."""
    if ending == "escaped":
        return f"{name} opened the vault, claimed the power-up, and escaped!"
    elif ending == "ran_out_of_time":
        return f"{name} ran out of time in the dungeon."
    return f"{name} left the dungeon without escaping."


def ending_page(state: State, ending: str):
    """Show the ending, score, and option to play again."""
    return Page(
        H1("The End"),
        P(ending_text(ending, state.player_name)),
        P(f"Final score: {final_score(state.inventory)}"),
        Button("Play again", "/restart"),
    )


# Routes
```

```
@route
def index(state: State):
    """Show the start screen."""
    return Page(
        H1("Dungeon Escape"),
        P(START_MESSAGE),
        TextBox("name"),
        Button("Start game", start_game),
    )


@route
def start_game(state: State, name: str):
    """Start a new game with the player's name."""
    state.player_name = name
    state.hours_left = START_HOURS
    state.dungeon_room = START_ROOM
    state.money = START_MONEY
    state.inventory = []
    state.message = f"Good luck, {name}!"
    state.ending = ""
    return explore(state)


@route
def time_left(state: State):
    """Show remaining time and whether the game is over."""
    if state.hours_left <= 0:
        state.ending = pick_ending(state)
        return ending_page(state, state.ending)
    return Page(P(f"Hours left: {state.hours_left}"))


@route
def explore(state: State):
    """Show the current room and room choices."""
    return Page(
        H1(state.dungeon_room),
        P(room_description(state.dungeon_room)),
        P(state.message),
        P(f"Hours left: {state.hours_left}"),
        P(f"Coins: {state.money}"),
        *[
```

```
            Button(room, loot_room, room)
            for room in room_lines()
        ],
        Button("View inventory", view_items),
        Button("Leave dungeon", leave_dungeon),
    )


@route
def loot_room(state: State, take_item: str):
    """Explore a room, spend time, and collect its item and coins."""
    cost = room_cost(take_item)

    if state.hours_left < cost:
        state.message = "You don't have enough time to explore that room."
    else:
        state.hours_left -= cost
        state.dungeon_room = take_item
        state.money += room_coins(take_item)

        item = make_item(take_item)
        if not has_item(state.inventory, item.name):
            state.inventory.append(item)
            state.message = f"You found {item.name} and some coins!"
        else:
            state.message = "You've already found this room's item."

    if state.hours_left <= 0:
        state.ending = pick_ending(state)
        return ending_page(state, state.ending)

    return explore(state)


@route
def view_items(state: State):
    """Show the player's inventory."""
    return Page(
        H1("Inventory"),
        *[P(line) for line in inventory_lines(state.inventory)],
        P(f"Coins: {state.money}"),
        Button("Back to exploring", explore),
    )
```

```
@route
def leave_dungeon(state: State):
    """End the game when the player chooses to leave."""
    state.ending = pick_ending(state)
    return ending_page(state, state.ending)


@route
def finale(state: State):
    """Show the final score and ending."""
    state.ending = pick_ending(state)
    return ending_page(state, state.ending)


@route
def restart(state: State):
    """Reset the game."""
    state.player_name = ""
    state.hours_left = START_HOURS
    state.dungeon_room = START_ROOM
    state.money = START_MONEY
    state.inventory = []
    state.message = START_MESSAGE
    state.ending = ""
    return index(state)
```

A few parts depend on the rules you decide:

I used "Vault Key" as an example requirement in pick_ending; your current make_item function doesn't create one yet.
I treated coins and item value as separate things: money is coins, while final_score totals inventory value.
You'll probably want a dedicated vault-opening action that checks state.money >= VAULT_COST, subtracts the cost, and grants the power-up.

should i use assert equals for my code?

Yes—use assert checks to test whether your helper functions return the values you expect. Python's assert statement is the usual choice; there isn't a built-in assert_equals statement.

For example:

```
assert room_cost("Crystal Cave") == 2
assert room_coins("Goblin Den") == 8
```

```
items = [
    Item("Glowing Crystal", "Hums with soft light.", 25),
    Item("Goblin Dagger", "Chipped but still sharp.", 10),
]

assert has_item(items, "Glowing Crystal") is True
assert has_item(items, "Dragon Egg") is False
assert total_loot_value(items) == 35
assert count_power_ups(items) == 1
assert final_score(items) == 40  # 35 loot + 5 power-up bonus
```

Put these in a separate test file or run them after your function definitions while you're learning. Don't put them inside the Drafter routes, because then they'll run as part of the game. Also, assert is best for development checks; if you later build a larger project, a test framework like pytest gives you more detailed results.

i have to put this into github. is there anything i should add so the code runs in github or will it run just the way it is

GitHub stores your code, but it won't automatically run a Drafter game just because you upload it. Your program also needs the required files and a place to run it.

For a basic upload, include:

Your Python game file, such as game.py
A requirements.txt file listing Drafter, for example:
drafter
A README.md with instructions for installing and starting the game, such as python game.py—though the exact command depends on how your class set up Drafter.
Before uploading, check that your code is complete and properly indented. The code in your first message lost its indentation when pasted, so it won't run in that form. Also check that every route uses Drafter's expected syntax and that State has any fields your routes use.

If your teacher only wants the code submitted to GitHub, that may be enough. If they want people to play it from a link, you'll need to deploy it to a host that supports Python and Drafter. I'd check your class instructions for the required launch command or hosting service.

how do I create a repository and put my code in it

Here's a straightforward way to put your game on GitHub:

Make sure your code is saved in a file on your computer, such as game.py.

Create a repository: Sign in at GitHub, click + in the top-right, then New repository. Give it a name, such as dungeon-escape-game, choose Public or Private, then click Create repository. If your code is already on your computer, leave “Add a README” unchecked for now.

Upload your code: On the new repository page, choose Add file → Upload files. Drag in game.py and any other files your project needs, such as requirements.txt or README.md.

Save the upload: Add a short message like Add dungeon game, then click Commit changes.

Your code will now be stored in the repository. GitHub stores and displays it; it won’t run the Drafter game automatically. If you want, tell me whether you’re using the GitHub website or GitHub Desktop, and I can give you the exact steps for that method.

Pasted Code

create the starting page



create crystal cave



This image depicts the 'Crystal Cave' from your dungeon. Note the large, pulsing crystals lining the walls and the specific 'Glowing Crystal' (the loot item) embedded in the rock formation on the left, humming with internal light. The rugged stone ground holds the small cluster of coins also found here.

create goblin den



Create flooded tunnel



Create the vault



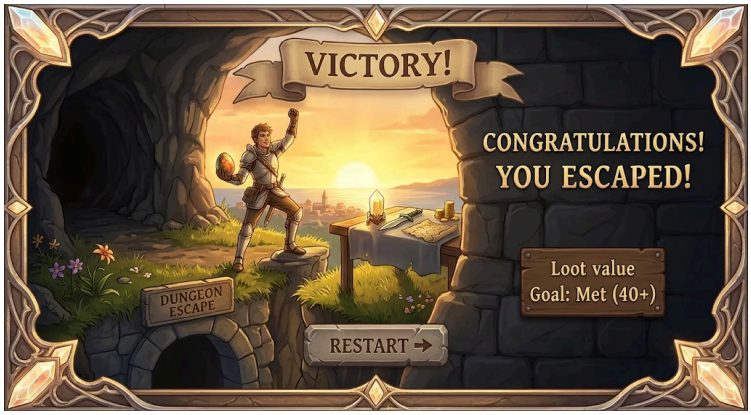
create a fail page

I created a distinct Game Over (Fail) page for when you run out of time, showing the tunnel collapsing, blocking the exit, and leaving you trapped. This uses a dark fantasy storybook illustration style to match the other scenes while focusing on the theme of being trapped and running late.

You can view the generated images below:



Create the victory/escape page

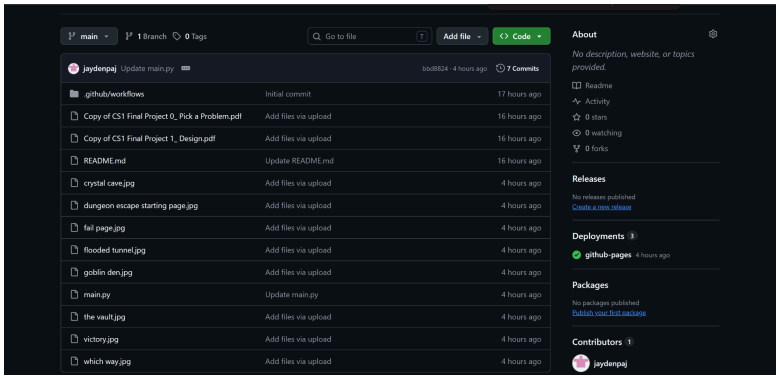


make a dungeon with 4 paths to choose from



give me tips on how to add pictures to my website/code with these jpgs, but don't just do it for me

Pasted Code



Your JPGs are already in the top level of your GitHub repository, so Drafter should be able to serve them from the site. The main idea is to put an image element in the Page returned by a route, and give it the image's repository filename.

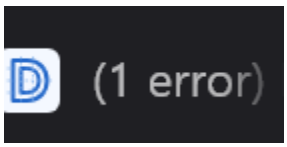
For example, Drafter pages often use Image(...). Try adding one to a page's list of contents:

```
return Page(state, [
    Header("Dungeon Escape"),
    Image("dungeon escape starting page.jpg"),
    # the rest of the page contents...
])
```

A few tips as you add them:
- Start with one image on the start screen, then run the game and see whether it appears.
- Use the filename exactly as it appears in GitHub, including spaces and capitalization. For example, "crystal cave.jpg" and "Crystal Cave.jpg" are different paths.
- Match images to routes: the starting-page image in index, room pictures in the room page in choose_room, and victory.jpg or fail page.jpg on the ending page.
- Keep the images in the repository alongside main.py, as they are now. If you later put them in a folder such as images, include that folder in the path, like "images/crystal cave.jpg".
- If an image looks too large, check Drafter's Image options in your class materials or documentation for setting its displayed size.
You can use the same idea for a room image by choosing its filename based on the room. For instance, inside choose_room, create a variable for the picture after the cave value has been checked. The exact mapping might be:

```
# Example mapping to plan out:
# "Crystal Cave" -> "crystal cave.jpg"
# "Goblin Den" -> "goblin den.jpg"
# "Flooded Tunnel" -> "flooded tunnel.jpg"
# "Vault" -> "the vault.jpg"
```

Then pass that filename to Image(...) in the page content. Try writing that small mapping yourself and test each room.



Pasted Code 2

my code says 1 error when i run it. everything works fine though so is there a need to fix it? also, where do i see what the error is?

If the game runs and behaves correctly, the "1 error" may be coming from a test or check rather than from the game itself. It's still worth finding out what it is, especially before submitting, since an error could point to a problem that only appears in a less common situation.

Click the "1 error" indicator in the place where you ran the code. It should open an error or output panel with details—often including the error message and a line number. If you're using an online class environment, look for a Console, Output, Problems, or Tests panel.

The screenshot only shows the error count, so I can’t tell what failed from it alone. If you paste the message or share a screenshot of the details, I can help you understand whether it needs fixing.